



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

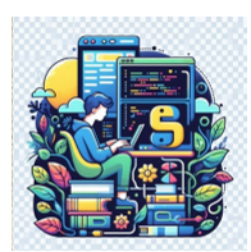
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Introspection

Lekha A

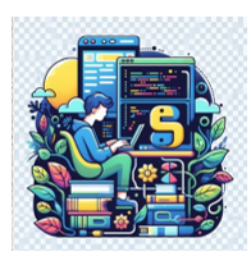
Computer Applications



PROGRAMMING WITH PYTHON

Introspection

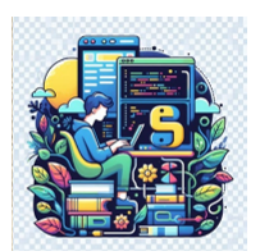
- Introspection is a technique to determine the type of an object at runtime.
- It is a code that inspects other modules and functions in memory as objects, getting information about them and manipulating them.
- It provides the facility to get familiar with an object's properties and attributes.
- By using introspection, Python objects can be dynamically inspected.



PROGRAMMING WITH PYTHON

Introspection

- Python is a dynamic, object-oriented, introspection support language.
- The introspection feature makes it a more powerful language than others.
- Python provides several functions and utilities for code introspection.



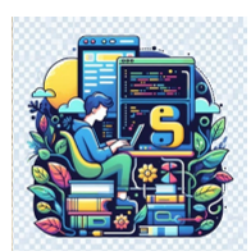
PROGRAMMING WITH PYTHON

Introspection

- The `dir()` function returns a sorted list of the attributes and methods belonging to an object.

```
myList = [1,2,3,4,5]  
print(dir(myList))
```

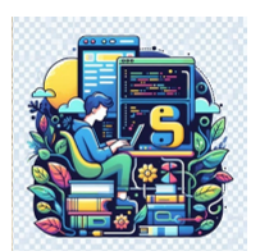
```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```



PROGRAMMING WITH PYTHON

Introspection

- ```
print("The methods and attributes is used with integer: ")
num = 5
print(dir(num))
print("The methods and attributes is used with string: ")
string = 'Python Programming'
print(dir(string))
```



# PROGRAMMING WITH PYTHON

## Introspection

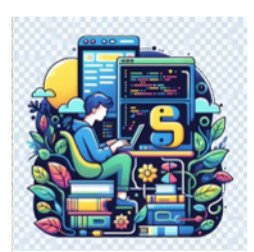
- The `type()` function returns the type of the object.

```
num = 15
print(type(num))

string = 'PythonProgamming'
print(type(string))
```

```
<class 'int'>
```

```
<class 'str'>
```



# PROGRAMMING WITH PYTHON

## Introspection

- The type() function returns the type of the object.

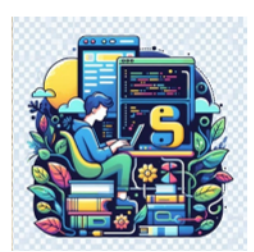
```
import sys
def function():
 pass

class MyClass(object):

 def __init__(self):
 pass

obj = MyClass()
print(type(1))
print(type(""))
print(type([]))
print(type({}))
print(type(()))
print(type(object))
print(type(function))
print(type(MyClass))
print(type(obj))
print(type(sys))
```

```
<class 'int'>
<class 'str'>
<class 'list'>
<class 'dict'>
<class 'tuple'>
<class 'type'>
<class 'function'>
<class 'type'>
<class '__main__.MyClass'>
<class 'module'>
```



# PROGRAMMING WITH PYTHON

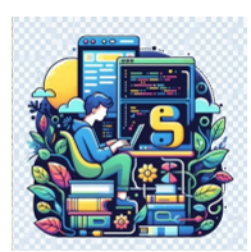
## Introspection

- The `hasattr()` function checks if the object has an attribute.
- Depending upon the result, it returns either true or false if the object has a given attribute.

```
class student:
 name = "Tony"
 age = 25
obj = student()
print(hasattr(obj, 'name'))
print(hasattr(obj, 'gpa'))
```

True

False



# PROGRAMMING WITH PYTHON

## Introspection

- The `id()` function returns the special id of an object.

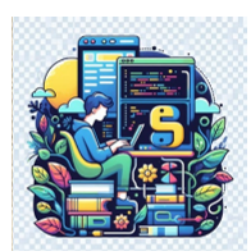
```
import sys
def function():
 pass

class MyClass(object):

 def __init__(self):
 pass

obj = MyClass()
print("The integer id is: ", id(1))
print("The string id is: ", id(""))
print("The list id is: ", id([]))
print("The set id is: ", id({}))
print("The list id is: ", id(()))
print("The object id is: ", id(object))
print("The function id is: ", id(function))
print("The MyClass id is: ", id(MyClass))
print("The obj id is: ", id(obj))
print("The sys module id is: ", id(sys))
```

The integer id is: 140724375308728  
The string id is: 140724375330048  
The list id is: 2043368555776  
The set id is: 2043368414848  
The list id is: 140724375381688  
The object id is: 140724374148464  
The function id is: 2043362694048  
The MyClass id is: 2043324037312  
The obj id is: 2043362156208  
The sys module id is: 2043267513488



# PROGRAMMING WITH PYTHON

## Introspection

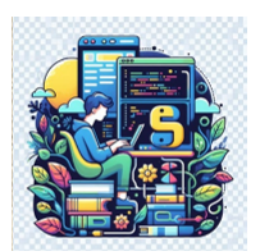
- **vars()**
  - It contains all instance variable (name and value) as a dictionary.
  - It is equivalent to mycar.\_\_dict\_\_.\*

```
class Car:
 def __init__(self, brand, model):
 self.brand = brand
 self.model = model

Create an instance of Car
mycar = Car("Toyota", "Camry")

Access instance variables as a dictionary
print(vars(mycar))

{'brand': 'Toyota', 'model': 'Camry'}
```



# PROGRAMMING WITH PYTHON

## Introspection

- **isinstance()**
  - It checks if an object is of a specified class.

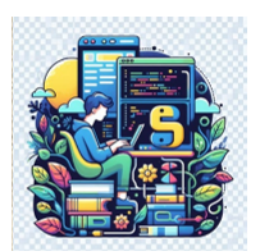
```
class Vehicle:
 pass

class Car(Vehicle):
 pass

mycar = Car()
print(isinstance(mycar, Car))
print(isinstance(mycar, Vehicle))
```

True

True



# PROGRAMMING WITH PYTHON

## Introspection

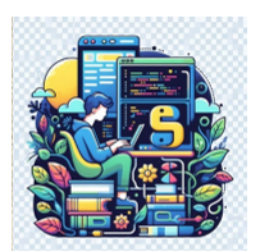
- **getsizeof()**
  - It is a part of the sys module; it contains the size of the object in bytes.

```
import sys
a = 10
b = "Hello, World!"
c = [1, 2, 3, 4, 5]
print(sys.getsizeof(a))
print(sys.getsizeof(b))
print(sys.getsizeof(c))
```

28

54

104



# PROGRAMMING WITH PYTHON

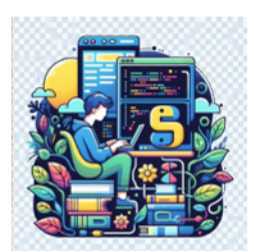
## Introspection

- **help()**
  - It obtains help on an object's interface based on docstrings.

```
class Car:
 """Car class represents a vehicle with a brand and model."""
 def __init__(self, brand, model):
 """Initialize the car with brand and model."""
 self.brand = brand
 self.model = model
help(Car)
```

Help on class Car in module \_\_main\_\_:

```
class Car(builtins.object)
| Car(brand, model)
|
| Car class represents a vehicle with a brand and model.
|
| Methods defined here:
|
| __init__(self, brand, model)
| Initialize the car with brand and model.
|
| -----
| Data descriptors defined here:
|
| __dict__
| dictionary for instance variables (if defined)
|
| __weakref__
| list of weak references to the object (if defined)
```



# PROGRAMMING WITH PYTHON

## Introspection

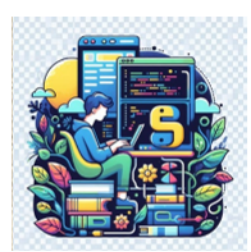
- **callable()**
  - It checks whether or not an object is callable.

```
class Car:
 def start(self):
 print("Car started!")
mycar = Car()
print(callable(Car))
print(callable(mycar.start))
print(callable(mycar))
```

True

True

False



# PROGRAMMING WITH PYTHON

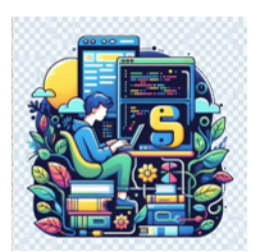
## Introspection Attributes

- Some attributes that give useful information about objects.
- **\_\_name\_\_**
  - It shows the original name of the class, function or method.

```
class Car:
 def start(self):
 print("Car started!")

print(Car.__name__)
print(Car.start.__name__)
```

```
Car
start
```



# PROGRAMMING WITH PYTHON

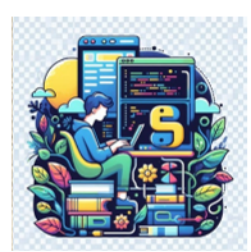
## Introspection Attributes

- `__qualname__`
  - The qualified name of the class function or method.
  - It is often useful where class/function/method definitions are nested.

```
class Vehicle:
 class Car:
 def start(self):
 print("Car started!")
print(Vehicle.Car.__qualname__)
print(Vehicle.Car.start.__qualname__)
```

Vehicle.Car

Vehicle.Car.start



# PROGRAMMING WITH PYTHON

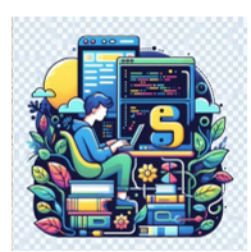
## Introspection Attributes

- `__doc__`
  - It is a documentation string, which can also be retrieved by calling built-in

function **help()**

```
class Car:
 """Car class represents a vehicle."""
 def start(self):
 """Start the car."""
 print("Car started!")
print(Car.__doc__)
print(Car.start.__doc__)
```

```
Car class represents a vehicle.
Start the car.
```



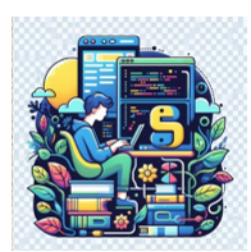
# PROGRAMMING WITH PYTHON

## Introspection Attributes

- `__self__`
- It is a instance to which a method is found.

```
class Car:
 def start(self):
 print("Car started!")
mycar = Car()
bound_method = mycar.start
print(bound_method.__self__)
```

```
<__main__.Car object at 0x000001DBBF09AD80>
```



# PROGRAMMING WITH PYTHON

## Recap

---

- Introspection
  - Methods
  - Attributes



**THANK YOU**

---

**Lekha A**  
Computer Applications